Promot 1:/plan

You are building SevaFlow, an inclusive hospital queue-management website for a hackathon.

We have a TOTAL development window of 3 hours.
Use free tools and avoid paid services.
This is a fictional-data prototype, not a production clinical system.

Read the attached Website PRD if available.

TECH STACK
- React + TypeScript + Vite.
- Tailwind CSS for styling.
- Supabase for database and authentication.
- Minimal dependencies.
- A frontend build that can be deployed to Vercel.
- No AI API is needed at runtime.

PRODUCT ROLES
- Patient
- Caregiver
- Receptionist
- Doctor
- Hospital administrator
- Government demonstration analyst

CORE FLOW
Registration -> department selection -> automatic token ->
Waiting -> call next -> In Progress -> Completed.

RULES
- Prioritize a working core over decorative features.
- Use fictional patients only.
- Enforce permissions on the backend.
- Never expose secret or service-role keys.

- Never disable security to fix an error.
- Do not modify unrelated files.
- Do not delete existing work or deploy without my approval.
- Clearly distinguish Working, Simulated and Not Implemented.
- Do not silently substitute mock data for database functionality.

Create:
1. A concise implementation plan.
2. A page and route map.
3. A database outline.
4. A phased task checklist.
5. Acceptance tests.
6. A list of assumptions and integration mocks.

Save project decisions in PROJECT_CONTEXT.md.
Save progress in TASKS.md.

Do not implement yet. Stop after the plan for review.

Promot 2:Implement the website structure from the approved plan.

ROUTES

Public:
/
/register
/display/:facilityId

Patient:
/patient
/patient/token/:visitId
/patient/permissions
/patient/feedback

Staff:
/staff/login

/reception
/doctor
/admin
/admin/alerts
/admin/audit
/admin/settings

Government demonstration:
/government-demo

Use shared layouts:
- Public header and footer.
- Patient navigation.
- Staff sidebar and top bar.
- Fullscreen public display.
- Restricted government demonstration layout.

REQUIREMENTS
- Configure routing and clear page titles.
- Provide loading, empty, error and not-found states.
- Create reusable components.
- Keep role-specific navigation separate.
- Clearly label placeholders.
- Do not claim access is protected until backend authorization exists.
- Do not implement every feature yet.

Start the development server and confirm all routes render.
List which pages are functional and which remain placeholders.

Promot 3:Improve the SevaFlow UI without changing its approved routes or business logic.

VISUAL DIRECTION
A polished, calm public-healthcare operations product.
Not a generic hospital advertisement.

COLOURS
- Primary teal: #0F766E
- Dark navy: #0F172A
- Page background: #F8FAFC
- Card background: white
- Use semantic colours with text labels for statuses.

LAYOUT
- Desktop staff sidebar around 240px wide.
- Clear top bar with page title, facility and connection status.
- Consistent spacing using an 8px-based scale.
- Restrained shadows and rounded cards.
- Readable tables and forms.
- No horizontal page overflow on mobile.
- Use a system font stack; do not depend on paid assets.

LANDING PAGE
- Short value proposition.
- Register / Track Token primary actions.
- Assisted-registration information.
- Language selection.
- Three concise benefit cards.
- No invented testimonials, certifications or impact statistics.

PATIENT DASHBOARD
- Large active-token card.
- Department and room.
- Current status.
- Patients ahead.
- Approximate waiting-time range.
- Last-updated timestamp.
- Clear next action.

RECEPTIONIST DASHBOARD
- Registration form.

- Department filter.
- Searchable queue table.
- Clearly separated primary actions.

DOCTOR DASHBOARD
- Current patient card.
- Next eligible patients.
- Call next, start and complete controls.
- Available / paused toggle.

ADMIN DASHBOARD
- Queue-count cards.
- Waiting-time metrics.
- Department queue table.
- Alerts and recent activity.

PUBLIC DISPLAY
- Large token and room text.
- Upcoming tokens.
- No personal contact details.
- Visual announcements that remain understandable without audio.

ACCESSIBILITY
- Keyboard-visible focus.
- Proper form labels.
- Scalable text.
- Status text, not colour alone.
- Accessible button names.

Use simple icons, not decorative stock imagery.
Avoid large animations.
Inspect desktop and narrow mobile layouts after implementation.

Promot 4:Implement the minimum secure Supabase backend for SevaFlow.

FIRST-PASS TABLES
- facilities
- departments
- profiles
- role_assignments
- visits
- tokens
- queue_events
- doctor_availability
- feedback

Add priority and audit fields/tables as required by the workflow.

REQUIREMENTS
- Provide reproducible SQL migrations.
- Use unique immutable internal IDs.
- Displayed tokens must be unique per facility, date and department.
- Use server-side timestamps.
- Prevent duplicate submissions with idempotency keys.
- Generate tokens transactionally.
- Enforce row-level access policies.

PERMISSIONS
- Patients access only their own visits.
- Staff access only authorized facility data.
- Doctors access authorized queues.
- Administrators manage their facility.
- Government demo users see synthetic aggregate data only.
- Users cannot assign themselves staff roles.
- Public display access must not expose private visit records.

AUTHENTICATION
Use Supabase authentication.
Seed staff roles through a protected setup process.
Do not invent a working SMS OTP provider.
Clearly label any demo authentication fallback.

SECRETS
Provide .env.example with variable names only.
Never place secret/service-role keys in browser code or Git.
Tell me which configuration I must enter manually.

Verify persistence and test unauthorized access.
Do not disable policies to make the UI work.

Promot 5:Implement and verify the complete SevaFlow patient journey.

1. Register a fictional patient.
2. Select an available department.
3. Create a unique token.
4. Show the visit as Waiting.
5. Authorized staff call the next eligible patient.
6. Doctor starts consultation: In Progress.
7. Doctor completes consultation: Completed.
8. Update patient, receptionist and doctor views.

RULES
- Call-next must execute atomically.
- Two concurrent requests must not call the same visit.
- Disable repeat actions while requests are pending.
- Preserve authoritative validation on the backend.
- Store called, started and completed timestamps separately.
- Show actionable errors.
- Support browser printing of a minimal token.
- Do not add advanced features until this passes.

Run or provide reproducible tests.
Confirm records remain after refresh.
Report failures honestly.

Promot 6:Preserve the working core.

Add these features in order:
1. Automatic queue refresh every 5 seconds.
2. Read-only public token display.
3. Last-updated and disconnected indicators.
4. Approximate waiting-time ranges.
5. English/Hindi toggle for main screens.
6. Large-text toggle.
7. Staff priority override with mandatory reason and audit.
8. Doctor available/paused status.
9. In-app delay and approaching-turn notices.

WAITING ESTIMATES
Use a documented configurable consultation-duration assumption
and the current eligible queue.
Clearly label estimates approximate.
Do not claim a validated predictive model.

SAFETY
No diagnosis or autonomous triage.
Emergency messaging directs users to clinical staff.
Do not expose medical reasons on public screens.

After each feature, verify the existing patient journey still works.
Stop and report if a change breaks registration or consultation.

Promot 7:Build lightweight supporting features without rewriting the core.

HOSPITAL ADMIN
- Waiting, In Progress and Completed counts.
- Department queue lengths.
- Waiting-time metrics calculated from stored events.
- Basic crowding threshold alert.
- Alert acknowledgement and response note.

- Priority and status-change audit list.
- Optional feedback review.

GOVERNMENT DEMONSTRATION
- Three fictional facilities.
- Synthetic aggregate queue load and service availability.
- Date/facility filters.
- Data-freshness labels.
- No identifiable patient drill-down.
- Visible "Synthetic demonstration data" label.

REMAINING FEATURES
- ABDM registration preview: explicitly simulated.
- SMS/WhatsApp delivery preview: explicitly simulated.
- Caregiver authorization: implement only if backend permissions and revocation can be completed and tested.
- If caregiver access is incomplete, label it a preview.
- Offline registrations must remain pending until server acceptance; never mint authoritative tokens locally.

Maintain a feature matrix:
Working / Simulated / Not Implemented.

Never invent metrics, deliveries or integration success.

Promot 8:Stop adding features. Verify the website.

Run:
- Type checking.
- Production build.
- Available automated tests.

Use the browser to test:
1. Register a patient.
2. View the generated token.
3. Call next from the staff view.

4. Start and complete consultation.
5. Verify the public display updates.
6. Refresh and confirm persistence.
7. Test a narrow mobile viewport.
8. Check loading, empty and error states.
9. Verify role restrictions.
10. Confirm simulated integrations are labelled.

Capture screenshots of:
- Patient tracker
- Receptionist dashboard
- Doctor dashboard
- Public display
- Administrator dashboard
- Government demo

Fix blocking defects first.
Do not redesign working pages.
Do not report a test as passed unless it was executed.

Produce a final walkthrough with:
- Setup instructions
- Test results
- Known issues
- Working/simulated/missing features
- Deployment instructions

Do not publish or deploy until I approve.